

# Déployer MLflow sur son EC2

Procédure complète et fichiers corrigés pour le serveur MLflow : image Docker dans ECR, déploiement automatique depuis GitHub, accès S3 par le rôle de l'instance, sans aucune clé AWS.

0 Vue d'ensemble

1 Fichiers

2 AWS

3 GitHub

4 Workflow

5 Déployer

6 Migrer l'existant

7 Dépannage

## 0 Ce qu'on déploie

Un serveur MLflow dans un conteneur Docker, sur un EC2 dédié. Les métadonnées (runs, métriques) sont dans une base SQLite sur un volume Docker, les artefacts dans `s3://nappecast/mlflow-artifacts`. Chaque push sur `master` qui touche `docker/mlflow/` reconstruit l'image et la redéploie automatiquement.



| Ressource          | Valeur                                                                                        |
|--------------------|-----------------------------------------------------------------------------------------------|
| Compte / région    | 844099234486 / eu-west-3                                                                      |
| Dépôt GitHub       | lauradmch/NappeCast-forecasting, branche master                                               |
| EC2                | Amazon Linux 2023, t3.small, utilisateur SSH ec2-user                                         |
| Adresses           | Elastic IP 51.45.56.7 (publique) · 172.31.32.79 (privée, utilisée par Airflow)                |
| Security Group     | sg-06cdd4979e8928cf2 : 22 et 5000 depuis vos IP, 5000 depuis le SG Airflow                    |
| Rôle de l'instance | mlflow-ec2-role : pull ECR + S3 mlflow-artifacts/*                                            |
| Dépôt ECR          | mlflow, tags immuables                                                                        |
| Rôle GitHub (OIDC) | github-actions-deploy, partagé avec Airflow                                                   |
| Secrets GitHub     | MLFLOW_EC2_HOST, MLFLOW_EC2_USER, MLFLOW_EC2_SSH_KEY, MLFLOW_SG_ID (environnement production) |

CE QUI CHANGE PAR RAPPORT À TES FICHIERS

- **Plus de Secrets Manager pour MLflow.** Rien n'est sensible dans sa configuration : les trois variables vont dans `mlflow.env`, et `deploy.sh` n'a plus besoin de lire un secret.
- **Plus de clé AWS.** Le serveur accède à S3 grâce au rôle de l'instance.
- `mlflow.env` **contient la config du serveur.** `MLFLOW_TRACKING_URI` et `MLFLOW_EXPERIMENT_NAME` sont des réglages du *client* : ils partent dans `airflow.env`.
- **Version de MLflow épinglée** ( 3.14.0 , la même que le client d'Airflow).
- `deploy.sh` **simplifié**, et le workflow se déclenche aussi quand on le modifie.

## À FAIRE TOUT DE SUITE : SUPPRIMER AWS\_CREATE\_SECRET.SH

Ce fichier contient la clé `AKIA4JCB4FK3GABT3PM6` en clair. Supprimez-le du dépôt. S'il a déjà été poussé sur GitHub, la clé reste dans l'historique Git : sa **suppression dans IAM** est la seule protection réelle.

```
BASH
git rm docker/mlflow/aws_create_secret.sh
aws iam list-access-keys --query 'AccessKeyMetadata[?AccessKeyId==`AKIA4JCB4FK3GABT3PM6`]' # répéter
avec --user-name si besoin
aws iam delete-access-key --user-name <UTILISATEUR> --access-key-id AKIA4JCB4FK3GABT3PM6
```

## 1 Les fichiers du dépôt

Quatre fichiers dans `docker/mlflow/`, plus le workflow. Ils sont copiés sur l'instance à chaque déploiement : on ne les modifie jamais à la main sur le serveur.

```
DOCKERFILE - docker/mlflow/Dockerfile
FROM python:3.11-slim

# Même version que le client MLflow utilisé par Airflow
RUN pip install --no-cache-dir "mlflow==3.14.0" boto3 psycpg2-binary

EXPOSE 5000

CMD mlflow server \
  --backend-store-uri "$MLFLOW_BACKEND_STORE_URI" \
  --default-artifact-root "$MLFLOW_ARTIFACT_ROOT" \
  --host 0.0.0.0 \
  --port 5000 \
  --workers 1 \
  --allowed-hosts "*" \
  --cors-allowed-origins "$CORS_ALLOWED_ORIGINS"
```

```
INI - docker/mlflow/mlflow.env
# Configuration du serveur MLflow (non secrète, versionnée dans Git).
# Accès S3 : par le rôle IAM de l'instance, aucune clé AWS ici.
MLFLOW_BACKEND_STORE_URI=sqlite:///mlflow-data/mlflow.db
MLFLOW_ARTIFACT_ROOT=s3://nappecast/mlflow-artifacts
CORS_ALLOWED_ORIGINS=https://mlflow.mapflowing.com
AWS_DEFAULT_REGION=eu-west-3
```

Sans guillemets autour des valeurs ni espace en fin de ligne. Ce fichier est versionné : n'y mettez jamais de mot de passe.

```
YAML - docker/mlflow/docker-compose.yml
services:
  mlflow:
    image: 844099234486.dkr.ecr.eu-west-3.amazonaws.com/mlflow:latest # tag remplacé par le workflow
    ports:
      - "5000:5000"
    volumes:
      - mlflow_data:/mlflow-data
    env_file:
      - mlflow.env
    restart: unless-stopped

volumes:
  mlflow_data:
```

Le tag `:latest` est remplacé par le SHA du commit au moment du déploiement. Laissez la ligne `image:` sous cette forme.

```
BASH - docker/mlflow/deploy.sh
#!/usr/bin/env bash
# Exécuté sur l'EC2 MLflow par le workflow GitHub (sudo /opt/mlflow/deploy.sh).
set -euo pipefail

APP_DIR=/opt/mlflow
REGION="eu-west-3"
ECR_REGISTRY="844099234486.dkr.ecr.eu-west-3.amazonaws.com"

cd "$APP_DIR"
for f in docker-compose.yml mlflow.env; do
  [ -f "$f" ] || { echo "ERREUR : $APP_DIR/$f manquant"; exit 1; }
done
rm -f .env.runtime .env # anciens fichiers, plus utilisés

aws ecr get-login-password --region "$REGION" \
  | docker login --username AWS --password-stdin "$ECR_REGISTRY"

docker compose pull
docker compose up -d --remove-orphans
docker image prune -f
docker compose ps
```

```
BASH - vérifications avant commit
chmod +x docker/mlflow/deploy.sh
git check-ignore -v docker/mlflow/mlflow.env # ne doit RIEN afficher
ls -b docker/mlflow/ # aucun nom ne doit commencer par "\ "
```

## 2 AWS : créer l'infrastructure (une seule fois)

Depuis votre poste, avec l'AWS CLI configurée en administrateur. Gardez le même terminal ouvert jusqu'à la fin : les variables servent d'une commande à l'autre.

```
BASH
export AWS_DEFAULT_REGION=eu-west-3
export MYIP=$(curl -s https://checkip.amazonaws.com)
```

### 2.1 Dépôt ECR

```
BASH
aws ecr create-repository --repository-name mlflow --image-tag-mutability IMMUTABLE
```

**2.2 Rôle de l'instance.** C'est lui qui donne au serveur l'accès à ECR et à S3, sans aucune clé.

```
BASH
cat > /tmp/ec2-trust.json <<'EOF'
{"Version": "2012-10-17", "Statement": [{"Effect": "Allow",
  "Principal": {"Service": "ec2.amazonaws.com"}, "Action": "sts:AssumeRole"}]}
EOF

cat > /tmp/mlflow-ec2.json <<'EOF'
{
  "Version": "2012-10-17",
  "Statement": [
    {"Sid": "EcrAuth", "Effect": "Allow", "Action": "ecr:GetAuthorizationToken", "Resource": "*"},
    {"Sid": "EcrPull", "Effect": "Allow",
      "Action": ["ecr:BatchGetImage", "ecr:GetDownloadUrlForLayer"],
      "Resource": "arn:aws:ecr:eu-west-3:844099234486:repository/mlflow"},
    {"Sid": "Artefacts", "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject", "s3:DeleteObject"],
      "Resource": "arn:aws:s3:::nappecast/mlflow-artifacts/*"},
    {"Sid": "ArtefactsList", "Effect": "Allow", "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::nappecast",
      "Condition": {"StringLike": {"s3:prefix": ["mlflow-artifacts/*"]}}}
  ]
}
EOF

aws iam create-role --role-name mlflow-ec2-role --assume-role-policy-document file:///tmp/ec2-trust.json
aws iam put-role-policy --role-name mlflow-ec2-role --policy-name mlflow-ec2 \
  --policy-document file:///tmp/mlflow-ec2.json
aws iam create-instance-profile --instance-profile-name mlflow-ec2-role
aws iam add-role-to-instance-profile --instance-profile-name mlflow-ec2-role --role-name mlflow-ec2-role
```

**2.3 Clé SSH et Security Group.** Le port 22 n'est ouvert qu'à votre IP : le workflow ajoute celle du runner GitHub pendant le déploiement, puis la retire.

```
BASH
aws ec2 create-key-pair --key-name mlflow-deploy --key-type ed25519 \
  --query KeyMaterial --output text > ~/.ssh/mlflow-deploy.pem
chmod 400 ~/.ssh/mlflow-deploy.pem

VPC_ID=$(aws ec2 describe-vpcs --filters Name=is-default,Values=true --query 'Vpcs[0].VpcId' --output text)
export SG_ID=$(aws ec2 create-security-group --group-name mlflow-sg --description "MLflow" \
  --vpc-id $VPC_ID --query GroupId --output text)
aws ec2 authorize-security-group-ingress --group-id $SG_ID --protocol tcp --port 22 --cidr $MYIP/32
aws ec2 authorize-security-group-ingress --group-id $SG_ID --protocol tcp --port 5000 --cidr $MYIP/32
echo $SG_ID
```

**2.4 L'instance.** Le User Data installe Docker, Compose et jq. `HttpPutResponseHopLimit=2` est indispensable : sans lui, le conteneur ne peut pas obtenir les droits du rôle et MLflow échoue sur S3 (`NoCredentialsError`).

```
BASH
cat > /tmp/userdata.sh <<'EOF'
#!/bin/bash
set -e
dnf install -y docker jq
systemctl enable --now docker
mkdir -p /usr/local/lib/docker/cli-plugins
curl -fsSL "https://github.com/docker/compose/releases/latest/download/docker-compose-linux-$(uname -m)" \
  -o /usr/local/lib/docker/cli-plugins/docker-compose
chmod +x /usr/local/lib/docker/cli-plugins/docker-compose
mkdir -p /opt/mlflow
EOF

export INSTANCE_ID=$(aws ec2 run-instances \
```

```
--image-id resolve:ssm:/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64 \
--instance-type t3.small --key-name mlflow-deploy --security-group-ids $SG_ID \
--iam-instance-profile Name=mlflow-ec2-role \
--metadata-options HttpTokens=required,HttpPutResponseHopLimit=2 \
--block-device-mappings 'DeviceName=/dev/xvda,Ebs={VolumeSize=20,VolumeType=gp3}' \
--user-data file:///tmp/userdata.sh \
--tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=mlflow}]' \
--query 'Instances[0].InstanceId' --output text)
aws ec2 wait instance-status-ok --instance-ids $INSTANCE_ID

# IP fixe (ne change pas si l'instance est arrêtée puis redémarrée)
ALLOC=$(aws ec2 allocate-address --domain vpc --query AllocationId --output text)
aws ec2 associate-address --instance-id $INSTANCE_ID --allocation-id $ALLOC
export EC2_HOST=$(aws ec2 describe-addresses --allocation-ids $ALLOC --query 'Addresses[0].PublicIp' --output text)

# IP privée, à reporter dans MLFLOW_TRACKING_URI de airflow.env
aws ec2 describe-instances --instance-ids $INSTANCE_ID \
--query 'Reservations[0].Instances[0].PrivateIpAddress' --output text
```

BASH – vérification

```
ssh -i ~/.ssh/mlflow-deploy.pem ec2-user@$EC2_HOST 'docker --version && docker compose version && jq --version'
```

### 3 GitHub : rôle OIDC et secrets

**3.1 Le rôle `github-actions-deploy`**, endossé par les workflows. Il est créé ici une seule fois, puis réutilisé par Airflow. Le fournisseur OIDC n'existe qu'une fois par compte :

```
BASH
aws iam list-open-id-connect-providers | grep -q token.actions.githubusercontent.com \
|| aws iam create-open-id-connect-provider --url https://token.actions.githubusercontent.com \
--client-id-list sts.amazonaws.com

cat > /tmp/gh-trust.json <<'EOF'
{"Version": "2012-10-17", "Statement": [{
  "Effect": "Allow",
  "Principal": {"Federated": "arn:aws:iam::844099234486:oidc-provider/token.actions.githubusercontent.com"},
  "Action": "sts:AssumeRoleWithWebIdentity",
  "Condition": {"StringEquals": {
    "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
    "token.actions.githubusercontent.com:sub": "repo:lauradmch/NappeCast-forecasting:environment:production"}}}]
EOF
aws iam create-role --role-name github-actions-deploy --assume-role-policy-document file:///tmp/gh-trust.json
```

Le `sub` se termine par `environment:production` parce que le job déclare `environment: production`. Si les deux ne correspondent pas, l'étape OIDC échoue avec `Not authorized to perform sts:AssumeRoleWithWebIdentity`.

**3.2 Les droits pour MLflow** : pousser l'image, vérifier si un tag existe, ouvrir et fermer le port 22 sur le Security Group de MLflow.

```
BASH
cat > /tmp/gh-mlflow.json <<EOF
{
  "Version": "2012-10-17",
  "Statement": [
    {"Sid": "EcrAuth", "Effect": "Allow", "Action": "ecr:GetAuthorizationToken", "Resource": "*"},
    {"Sid": "EcrPushMlflow", "Effect": "Allow",
     "Action": ["ecr:BatchCheckLayerAvailability", "ecr:InitiateLayerUpload", "ecr:UploadLayerPart",
               "ecr:CompleteLayerUpload", "ecr:PutImage", "ecr:BatchGetImage", "ecr:DescribeImages"],
     "Resource": "arn:aws:ecr:eu-west-3:844099234486:repository/mlflow"},
    {"Sid": "SshMlflow", "Effect": "Allow",
     "Action": ["ec2:AuthorizeSecurityGroupIngress", "ec2:RevokeSecurityGroupIngress"],
     "Resource": "arn:aws:ec2:eu-west-3:844099234486:security-group/$SG_ID"}
  ]
}
EOF
aws iam put-role-policy --role-name github-actions-deploy --policy-name github-actions-mlflow \
--policy-document file:///tmp/gh-mlflow.json
```

**3.3 Les secrets GitHub**, dans l'environnement `production` (Settings → Environments → production), ou avec la CLI `gh` :

```
BASH
R=lauradmch/NappeCast-forecasting
gh secret set MLFLOW_EC2_HOST --env production --repo $R --body "$EC2_HOST"
gh secret set MLFLOW_EC2_USER --env production --repo $R --body "ec2-user"
gh secret set MLFLOW_EC2_SSH_KEY --env production --repo $R < ~/.ssh/mlflow-deploy.pem
gh secret set MLFLOW_SG_ID --env production --repo $R --body "$SG_ID"
```

Un secret absent ne provoque pas d'erreur GitHub : il est remplacé par une chaîne vide. La première étape du workflow vérifie donc les quatre.

## 4 Le workflow

```
YAML — .github/workflows/cd-mlflow.yml
name: cd-mlflow

on:
  push:
    branches: [master]
    paths:
      - 'docker/mlflow/**'
      - '.github/workflows/cd-mlflow.yml'
  workflow_dispatch:

permissions:
  id-token: write
  contents: read

concurrency:
  group: cd-mlflow
  cancel-in-progress: false

env:
  SVC: mlflow
  SVC_DIR: docker/mlflow
  AWS_REGION: eu-west-3
  ECR_REGISTRY: 844099234486.dkr.ecr.eu-west-3.amazonaws.com
```

```

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4

      - name: Vérifier les fichiers et les secrets
        env:
          EC2_HOST: ${ secrets.MLFLOW_EC2_HOST }
          EC2_USER: ${ secrets.MLFLOW_EC2_USER }
          EC2_SSH_KEY: ${ secrets.MLFLOW_EC2_SSH_KEY }
          SG_ID: ${ secrets.MLFLOW_SG_ID }
        run: |
          ls -la "$SVC_DIR/"
          for f in Dockerfile docker-compose.yml "$SVC.env" deploy.sh; do
            test -f "$SVC_DIR/$f" || { echo "::error::$SVC_DIR/$f absent du commit"; exit 1; }
          done
          for v in EC2_HOST EC2_USER EC2_SSH_KEY SG_ID; do
            test -n "${!v}" || { echo "::error::secret MLFLOW_$v non défini"; exit 1; }
          done

      - name: Auth AWS via OIDC
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::844099234486:role/github-actions-deploy
          aws-region: ${ env.AWS_REGION }

      - name: Login ECR
        uses: aws-actions/amazon-ecr-login@v2

      - name: L'image existe-t-elle déjà ?
        id: img
        run: |
          if aws ecr describe-images --repository-name "$SVC" \
            --image-ids imageTag="$GITHUB_SHA" >/dev/null 2>&1; then
            echo "exists=true" >> "$GITHUB_OUTPUT"
          else
            echo "exists=false" >> "$GITHUB_OUTPUT"
          fi

      - name: Set up Docker Buildx
        if: steps.img.outputs.exists == 'false'
        uses: docker/setup-buildx-action@v3

      - name: Build et push de l'image
        if: steps.img.outputs.exists == 'false'
        uses: docker/build-push-action@v6
        with:
          context: docker/mlflow
          file: docker/mlflow/Dockerfile
          push: true
          tags: ${ env.ECR_REGISTRY }/${ env.SVC }:${ github.sha }
          cache-from: type=gha
          cache-to: type=gha,mode=max

      - name: Injecter le tag dans docker-compose.yml
        run: |
          sed -i "s|image: $ECR_REGISTRY/$SVC:[^ ]*|image: $ECR_REGISTRY/$SVC:$GITHUB_SHA|" "$SVC_DIR/docker-
compose.yml"
          grep -q "$SVC:$GITHUB_SHA" "$SVC_DIR/docker-compose.yml" \
            || { echo "::error::Tag non injecté dans docker-compose.yml"; exit 1; }

      - name: IP publique du runner
        id: ip
        run: |
          IP=$(curl -sf https://checkip.amazonaws.com | tr -d '[:space:]')

```

```

    test -n "$IP" || { echo "::error::IP du runner introuvable"; exit 1; }
    echo "ip=$IP" >> "$GITHUB_OUTPUT"

- name: Ouvrir SSH pour ce runner
  id: open_ssh
  env:
    SG_ID: ${ secrets.MLFLOW_SG_ID }
    RUNNER_IP: ${ steps.ip.outputs.ip }
  run: |
    aws ec2 authorize-security-group-ingress --group-id "$SG_ID" \
      --protocol tcp --port 22 --cidr "$RUNNER_IP/32"

- name: Préparer la clé SSH
  env:
    SSH_KEY: ${ secrets.MLFLOW_EC2_SSH_KEY }
  run: |
    install -m 700 -d ~/.ssh
    printf '%s\n' "$SSH_KEY" > ~/.ssh/deploy_key
    chmod 600 ~/.ssh/deploy_key

- name: Copier les fichiers sur l'EC2
  env:
    HOST: ${ secrets.MLFLOW_EC2_HOST }
    USER: ${ secrets.MLFLOW_EC2_USER }
  run: |
    set -e
    OPTS="-i $HOME/.ssh/deploy_key -o StrictHostKeyChecking=accept-new -o ConnectTimeout=20"
    ssh $OPTS "$USER@$HOST" "rm -rf /tmp/$SVC-deploy && mkdir -p /tmp/$SVC-deploy"
    scp $OPTS "$SVC_DIR/docker-compose.yml" "$SVC_DIR/$SVC.env" "$SVC_DIR/deploy.sh" \
      "$USER@$HOST:/tmp/$SVC-deploy/"
    ssh $OPTS "$USER@$HOST" "ls -la /tmp/$SVC-deploy"

- name: Déploiement
  env:
    HOST: ${ secrets.MLFLOW_EC2_HOST }
    USER: ${ secrets.MLFLOW_EC2_USER }
  run: |
    ssh -i "$HOME/.ssh/deploy_key" -o StrictHostKeyChecking=accept-new -o ConnectTimeout=20 \
      "$USER@$HOST" "SVC=$SVC bash -s" <<'EOF'
    set -e
    D=/tmp/$SVC-deploy
    APP=/opt/$SVC
    sudo mkdir -p "$APP"
    sudo install -m 644 "$D/docker-compose.yml" "$D/$SVC.env" "$APP/"
    sudo install -m 755 "$D/deploy.sh" "$APP/deploy.sh"
    sudo "$APP/deploy.sh"
    rm -rf "$D"
    EOF

- name: Refermer SSH
  if: always() && steps.open_ssh.outcome == 'success'
  env:
    SG_ID: ${ secrets.MLFLOW_SG_ID }
    RUNNER_IP: ${ steps.ip.outputs.ip }
  run: |
    aws ec2 revoke-security-group-ingress --group-id "$SG_ID" \
      --protocol tcp --port 22 --cidr "$RUNNER_IP/32"

- name: Supprimer la clé SSH du runner
  if: always()
  run: rm -f ~/.ssh/deploy_key

```

## CE QUE FAIT LE WORKFLOW

1. Vérifie que les fichiers et les 4 secrets existent.
2. Se connecte à AWS par OIDC (aucune clé AWS dans GitHub).
3. Construit et pousse l'image `mlflow:<sha>`, sauf si ce tag existe déjà (re-run).
4. Écrit ce tag dans `docker-compose.yml`.
5. Ouvre le port 22 pour l'IP du runner, copie les 3 fichiers, lance `deploy.sh`.
6. Referme le port 22 et supprime la clé SSH du runner, même en cas d'échec.

## 5 Déployer et vérifier

BASH

```
git add docker/mlflow .github/workflows/cd-mlflow.yml
git commit -m "mlflow: déploiement"
git push
```

Suivez le job dans l'onglet **Actions**. Pour redéployer sans commit : **Actions** → **cd-mlflow** → **Run workflow**. Évitez « Re-run » après une correction : il repart de l'ancien commit.

BASH

```
curl -s http://51.45.56.7:5000/health # doit afficher OK

ssh -i ~/.ssh/mlflow-deploy.pem ec2-user@51.45.56.7 '
  cd /opt/mlflow && sudo docker compose ps && sudo docker compose logs --tail 20 &&
  sudo docker compose exec mlflow python -c "import boto3; print(boto3.client(\"sts\").get_caller_identity()
[\"Arn\"])"'
```

La dernière commande doit afficher `...assumed-role/mlflow-ec2-role/i-...` : le conteneur utilise bien le rôle de l'instance, sans clé.

## 6 Migrer l'instance actuelle

L'instance en service (`i-06b315e78fa277e77`) utilise le rôle `mlflow-ec2-role2` et le secret `mlflow/prod-latest`. Pour passer aux nouveaux fichiers sans la recréer :

BASH

```
# 1. Droits S3 pour le rôle existant (remplace les clés AWS)
aws iam put-role-policy --role-name mlflow-ec2-role2 --policy-name mlflow-ec2 \
  --policy-document file:///tmp/mlflow-ec2.json # créez d'abord ce fichier (bloc cat de l'étape 2.2)

# 2. Autoriser les conteneurs à joindre le service de métadonnées
aws ec2 modify-instance-metadata-options --region eu-west-3 --instance-id i-06b315e78fa277e77 \
  --http-tokens required --http-put-response-hop-limit 2

# 3. Pousser les nouveaux fichiers : le workflow redéploie
git add docker/mlflow .github/workflows/cd-mlflow.yml && git commit -m "mlflow: config sans secret" && git push

# 4. Quand tout fonctionne : supprimer l'ancien secret et sa policy
aws secretsmanager delete-secret --region eu-west-3 --secret-id mlflow/prod-latest --recovery-window-in-days 7
aws iam delete-role-policy --role-name mlflow-ec2-role2 --policy-name GetSecretsManagerKey
```

À l'étape 4, la policy `GetSecretsManagerKey` contient aussi les droits ECR : ne la supprimez qu'après avoir vérifié que la nouvelle policy `mlflow-ec2` est bien attachée (elle les inclut).

## 7 Dépannage

| Symptôme                                                                        | Cause                                         | Solution                                                       |
|---------------------------------------------------------------------------------|-----------------------------------------------|----------------------------------------------------------------|
| <code>dial tcp ...:22: i/o timeout</code>                                       | Instance arrêtée, ou IP non autorisée         | <code>aws ec2 start-instances --instance-ids &lt;id&gt;</code> |
| <code>ssh: unable to authenticate</code>                                        | Mauvais utilisateur                           | <code>MLFLOW_EC2_USER = ec2-user</code>                        |
| <code>tag ... already exists ... immutable</code>                               | Build d'un commit déjà poussé                 | Géré par le workflow (build sauté)                             |
| <code>UnauthorizedOperation ec2:...</code><br><code>SecurityGroupIngress</code> | Policy 3.2 absente, ou mauvais ID de SG       | Refaire 3.2 avec le bon <code>SG_ID</code>                     |
| <code>--group-id: expected one argument</code>                                  | Secret <code>MLFLOW_SG_ID</code> absent       | Étape 3.3                                                      |
| <code>... mlflow.env</code> absent du commit                                    | Fichier ignoré par Git, ou espace dans le nom | <code>git check-ignore -v</code> , <code>ls -b</code>          |
| <code>NoCredentialsError</code> dans les logs                                   | Hop limit à 1, ou rôle sans droits S3         | Étape 6, points 1 et 2                                         |
| Airflow n'atteint pas <code>172.31.32.79:5000</code>                            | Port 5000 fermé au SG Airflow                 | Document Airflow, étape 2.4                                    |

Compte `844099234486`, région `eu-west-3`, dépôt `lauradmch/NappeCast-forecasting`. Document compagnon : « Déployer Airflow sur son EC2 ».